

Assignment 5

Part I: Loss Functions

Problem 1: Regression Loss

A neural network with 3 output dimensions produces prediction $\vec{y} = [2.1, -0.5, 1.8]$ for a training instance with ground truth $\bar{y} = [2.0, 0.0, 1.5]$.

- (a) Compute the regression (L2) loss $L = \|\vec{y} - \bar{y}\|^2$.
 - (b) Compute the gradient $\frac{\partial L}{\partial \vec{y}}$ of the regression loss with respect to the prediction vector.
 - (c) If the network has a single weight w feeding into the first output as $y_0 = w \cdot x_0$ with $x_0 = 3.0$, compute $\frac{\partial L}{\partial w}$ using the chain rule.
-

Problem 2: Cross-Entropy Loss

A 5-class classifier outputs the probability vector $\vec{y} = [0.05, 0.10, 0.60, 0.20, 0.05]$. The ground truth is class 2.

- (a) Write the one-hot ground truth vector $\bar{y}$.
- (b) Compute the cross-entropy loss $L = -\sum_j \bar{y}_j \log(y_j)$.
- (c) Suppose the classifier improves and now outputs $\vec{y}' = [0.01, 0.02, 0.92, 0.04, 0.01]$. Compute the new cross-entropy loss. By how much has the loss decreased?

(d) What is the minimum possible CE loss for this example? Under what prediction does it occur?

Problem 3: Binary Cross-Entropy

Consider binary classification where $\bar{y} = 1$ (GT is class 1).

(a) Compute the binary cross-entropy loss for predictions $y \in \{0.1, 0.3, 0.5, 0.7, 0.9, 0.99\}$.

(b) Plot (or sketch) the loss curve as a function of y for $y \in (0, 1)$ when $\bar{y} = 1$.

(c) Show analytically that the minimum of binary CE loss occurs when $y = \bar{y}$ by computing $\frac{\partial L}{\partial y}$ and setting it to zero.

Part II: Softmax

Problem 4: Softmax Computation

A 4-class classifier (cat=0, dog=1, airplane=2, auto=3) outputs score vector $\vec{s} = [3, 1, -1, 0]$.

(a) Compute the softmax probabilities $\text{softmax}(\vec{s})$.

(b) If the ground truth is class 0 (cat), compute the softmax cross-entropy loss.

(c) Now consider $\vec{s}' = [6, 2, -2, 0]$ (all scores doubled). Compute $\text{softmax}(\vec{s}')$. Is the network more or less confident? How does the CE loss compare?

(d) What happens to softmax as we apply temperature scaling $\text{softmax}(\vec{s}/\tau)$ with $\tau = 0.1$ vs. $\tau = 10$?

Problem 5: Softmax Properties

- (a) Prove that the softmax output sums to 1: $\sum_{j=0}^{N-1} \text{softmax}(\vec{s})_j = 1$.
- (b) Show that softmax is invariant to adding a constant: $\text{softmax}(\vec{s} + c\vec{1}) = \text{softmax}(\vec{s})$ for any scalar c . Explain why this property is useful for numerical stability.
- (c) Compute the Jacobian $\frac{\partial \text{softmax}(\vec{s})_i}{\partial s_j}$ for a general score vector.
-

Part III: Focal Loss and Hinge Loss

Problem 6: Focal Loss

Consider binary cross-entropy with ground truth class 1.

- (a) Write the focal loss formula $L = -(1 - y_t)^\gamma \log(y_t)$ for $\gamma = 0, 1, 2$.
- (b) Compute the focal loss for $y_t \in \{0.1, 0.5, 0.9\}$ at each value of γ .
- (c) For the "easy" example ($y_t = 0.9$), compute the ratio $\frac{L_{\text{focal}}(\gamma=2)}{L_{\text{CE}}}$. What fraction of the standard CE loss remains?
- (d) Explain how focal loss helps in a dataset where 95% of examples are class 0 and 5% are class 1.
-

Problem 7: Multi-class Hinge Loss

A 4-class classifier produces scores $\vec{y} = [5, 8, 3, 2]$ and the ground truth is class 1. Use margin $m = 1$.

- (a) Compute the hinge loss $L = \sum_{j \neq c} \max(0, y_j - y_c + m)$.

- (b) For each incorrect class, state whether it contributes to the loss and why.
- (c) What is the minimum score the correct class (class 1) must achieve for the hinge loss to be exactly zero?
- (d) Compare the behavior: if the classifier produces scores $[5, 100, 3, 2]$ vs. $[5, 8, 3, 2]$, what are the respective hinge losses? How does this differ from CE loss behavior?
-

Part IV: Optimization

Problem 8: Vanilla SGD

Consider loss function $L(w) = (w - 3)^2$ with initial weight $w_0 = 0$ and learning rate $\eta = 0.1$.

- (a) Compute the gradient $\frac{\partial L}{\partial w}$ as a function of w .
- (b) Perform 5 iterations of gradient descent. Record w_t and $L(w_t)$ at each step.
- (c) What value does w_t converge to as $t \rightarrow \infty$?
- (d) Repeat part (b) with $\eta = 1.0$. What happens? Repeat with $\eta = 1.1$. What happens?
-

Problem 9: Momentum

Using the same loss $L(w) = (w - 3)^2$, initial $w_0 = 0$, $\eta = 0.1$, momentum $\beta = 0.9$, and $\Delta w_{-1} = 0$:

- (a) Compute the momentum update: $\Delta w_t = \beta \Delta w_{t-1} + \eta \nabla_w L(w_t)$, then $w_{t+1} = w_t - \Delta w_t$. Perform 5 iterations.

(b) Compare your w_5 from momentum to w_5 from vanilla SGD (Problem 8b). Which converges faster?

(c) Explain what would happen if $\beta = 0$. What if $\beta = 0.99$?

Problem 10: Adam Optimizer

Suppose at iteration $t = 3$, Adam has accumulated $v_3 = 0.15$ and $s_3 = 0.04$ with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 0.001$, $\epsilon = 10^{-8}$.

(a) Compute the bias-corrected estimates $\hat{v}_3$ and $\hat{s}_3$.

(b) Compute the Adam update $\Delta w_3 = \eta \frac{\hat{v}_3}{\sqrt{\hat{s}_3 + \epsilon}}$.

(c) Why is bias correction important in early iterations? What values would $\hat{v}_3$ and $\hat{s}_3$ approach as $t \rightarrow \infty$?

Part V: Regularization and Dropout

Problem 11: L2 Regularization

A network has weight vector $\vec{w} = [3, -2, 1, 0.5]$ and bias $\vec{b} = [0.1, -0.2]$. The unregularized training loss is $L = 2.5$, and $\lambda = 0.01$.

(a) Compute the L2 regularization penalty $R(\theta) = \|\vec{w}\|^2 + \|\vec{b}\|^2$.

(b) Compute the total regularized loss $L_{\text{total}} = L + \lambda R(\theta)$.

(c) Compute $\frac{\partial R}{\partial w_0}$. How does the gradient of L2 regularization act on $w_0 = 3$?

(d) If we now use $\lambda = 1.0$, what is L_{total} ? Explain why using too large a λ is harmful.

Problem 12: L1 vs. L2 Regularization

Consider a single weight w with unregularized loss $L(w) = (w - 0.5)^2$ and $\lambda = 0.1$.

- (a) Write the total loss for L1 regularization: $L_1(w) = (w - 0.5)^2 + 0.1|w|$.
 - (b) Write the total loss for L2 regularization: $L_2(w) = (w - 0.5)^2 + 0.1w^2$.
 - (c) Find the optimal w^* analytically for L2 by setting $\frac{dL_2}{dw} = 0$.
 - (d) Show that L1 regularization can push w^* to exactly zero for sufficiently large λ . Find the critical λ value.
-

Problem 13: Dropout

A hidden layer has 4 neurons with outputs $[2.0, -1.0, 0.5, 3.0]$. Dropout probability is $p = 0.5$ (probability of keeping each neuron).

- (a) How many distinct subnetworks does dropout simulate for this layer?
 - (b) If the dropout mask is $[1, 0, 1, 0]$ (1 = keep, 0 = drop), what is the output after dropout during training?
 - (c) During inference (no dropout), the raw outputs are $[2.0, -1.0, 0.5, 3.0]$. Explain why we multiply by $p = 0.5$ or, equivalently, why PyTorch divides by p during training.
 - (d) Compute the expected output of each neuron during training with dropout. Verify it matches the inference output (scaled appropriately).
-

Problem 14: Bayesian Interpretation

- (a) Starting from Bayes' theorem $p(\theta|T) \propto p(T|\theta) p(\theta)$, show that MAP estimation with a Gaussian prior $p(\theta) \propto e^{-\lambda \|\theta\|^2}$ leads to the L2-regularized loss minimization.
- (b) What prior distribution corresponds to L1 regularization? Write $p(\theta)$ and identify the distribution by name.
- (c) In practical terms, what does choosing a large λ in MAP estimation imply about our prior belief regarding the magnitude of θ ?
-

Problem 15: Putting It All Together

You are training a 3-class classifier on an imbalanced dataset (class 0: 1000 samples, class 1: 50 samples, class 2: 50 samples). The network has two hidden layers of 128 neurons each.

- (a) Which loss function would you choose: MSE, CE, or focal loss? Justify your answer.
- (b) Which optimizer would you start with? Specify the hyperparameters.
- (c) The training loss decreases to near zero but test accuracy is only 60%. Diagnose the problem.
- (d) Propose two specific regularization techniques to address the problem from part (c), and explain how each helps.
- (e) Write a PyTorch code snippet implementing your chosen loss, optimizer, and regularization.